

ORACLE

JDK 25 – LTS y JDK 26

Aurelio Garcia-Ribeyro

Director Senior de Product Management

Disculpas a los puristas

Esta presentación es en Spanglish

Esta sesión fue escrita originalmente en inglés. La traduje con ayuda de inteligencia artificial, pero pedí que no tradujera:

- Términos técnicos que no tengan una traducción consistente a través de América Latina y España
- Nombres de los JEPs, incluyendo indicadores como "Preview" e "Incubator"
- Nombres de licencias

2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033

2025-09-16
2025-W38-2

28

27

2026-03-17
2026-W12-2

26

25 LTS

24

23

22

21 LTS

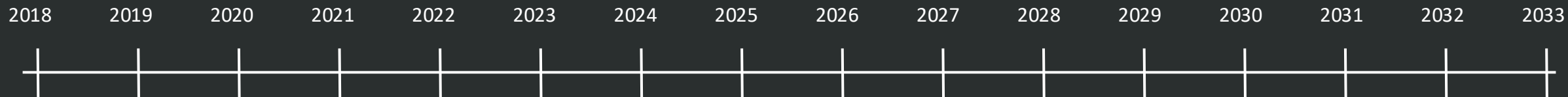
17 LTS

11 LTS

8

Oracle JDK

para clientes

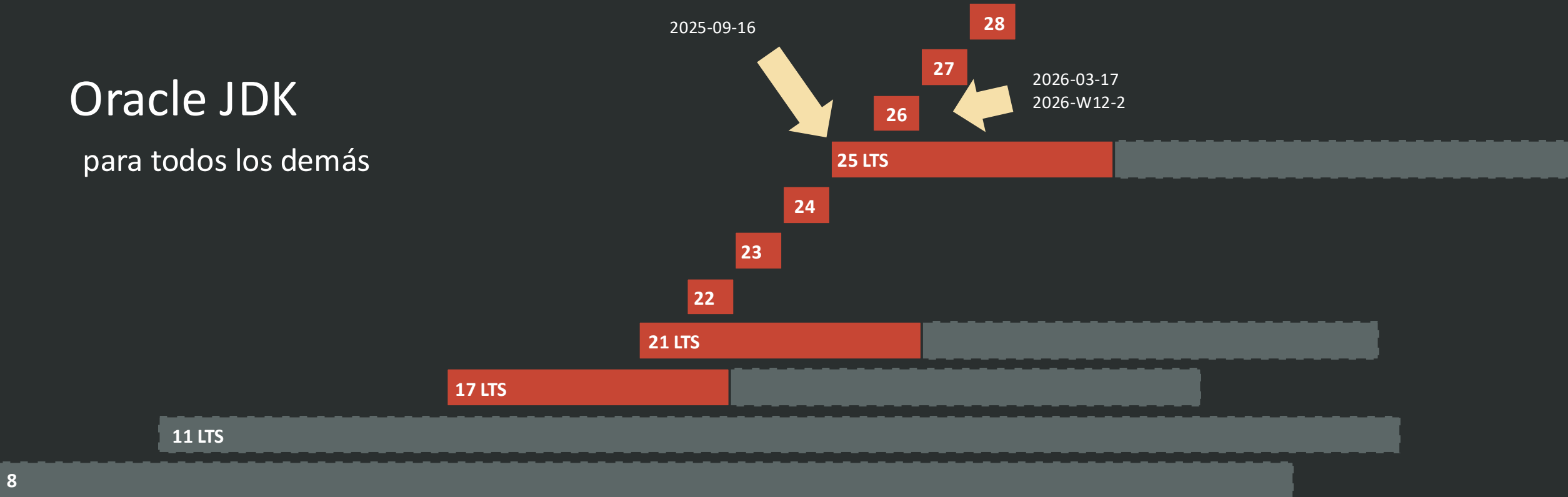


Oracle JDK

para todos los demás

2025-09-16

2026-03-17
2026-W12-2



Free Use Java License

Gratis para desarrollo y uso personal; Suscripción u OCI para producción



Novedades en JDK 25

Lenguaje

Primitive Types in Patterns, instanceof, and switch ^{3rd Preview} [507]

Module Import Declarations [511]

Compact Source Files and Instance Main Methods [512]

Flexible Constructor Bodies [513]

Performance

Compact Object Headers [519]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]

Ahead-of-Time Method Profiling [515]

Librerías

Structured Concurrency ^{5th Preview} [505]

Scoped Values [506]

Stable Values ^{Preview} [502]

Vector API ^{10th Incubator} [508]

Librerías de Seguridad

Key Derivation Function API [510]

PEM Encodings of Cryptographic Objects ^{Preview} [470]

Monitoreo

JFR CPU-Time Profiling ^{Experimental} [509]

JFR Cooperative Sampling [518]

JFR Method Timing & Tracing [520]

Novedades en JDK 25

Lenguaje

Primitive Types in Patterns, instanceof, and switch ^{3rd Preview} [507]

Module Import Declarations [511]

Compact Source Files and Instance Main Methods [512]

Flexible Constructor Bodies [513]

Primitive Types in Patterns, instanceof, and switch (3rd Preview)

JEP 507

Mejorar pattern matching al permitir tipos primitivos en todos los contextos de patrones y extender instanceof y switch para que funcionen con todos los tipos primitivos.

Pattern Matching for switch

```
switch (x.getStatus()) {  
    case 0 -> "okay";  
    case 1 -> "warning";  
    case 2 -> "error";  
    default -> "unknown status: " + x.getStatus();  
}
```


Pattern Matching for switch

```
switch (x.getStatus()) {  
    case 0 -> "okay";  
    case 1 -> "warning";  
    case 2 -> "error";  
    case int i -> "unknown status: " + i;  
}
```

Pattern Matching for switch

```
switch (x.getYearlyFlights()) {  
    case 0 -> ... ;  
    case 1 -> ... ;  
    case 2 -> issueDiscount();  
    case int i when i >= 100 -> issueGoldCard();  
    case int i -> ... appropriate action when  
                        i > 2 && i < 100 ...;  
}
```

Pattern Matching for instanceof

```
if (i >= -128 && i <= 127) {  
    byte b = (byte)i;  
    ... b ...  
}
```

Pattern Matching for instanceof

```
if (i instanceof byte b) {  
    ... b ...  
}
```

```
// if i doesn't "fit" in a byte  
// there is no cast
```

Module Import Declarations

JEP 511

Capacidad de importar de forma concisa todos los paquetes exportados por un módulo.

Algunas clases (como Object, String y Comparable) son esenciales para todo programa Java, por lo que el compilador las importa automáticamente como si cada archivo fuente tuviera

```
import java.lang.*;
```

al inicio.

Clases más recientes como List, Map, Stream y Path son casi igual de esenciales, pero no se importan automáticamente y requieren varias importaciones. Esta feature permite importar todas estas clases con una sola declaración import.

Module Import Declarations

```
String[] fruits = new String[] { "apple", "berry", "citrus" };  
Map<String, String> m =  
    Stream.of(fruits)  
        .collect(Collectors.toMap(s -> s.toUpperCase().substring(0,1),  
                                Function.identity()));
```

*No intenten entender este código.
Ese no es el punto...*

Module Import Declarations

```
import java.util.Map; // or import java.util.*;
import java.util.function.Function; // or import java.util.function.*;
import java.util.stream.Collectors; // or import java.util.stream.*;
import java.util.stream.Stream; // (can be removed)
```

```
String[] fruits = new String[] { "apple", "berry", "citrus" };
Map<String, String> m =
    Stream.of(fruits)
        .collect(Collectors.toMap(s ->s.toUpperCase().substring(0,1),
                                Function.identity()));
```

Pero desde JDK 9

... tenemos módulos

Que agrupan paquetes relacionados

Module Import Declarations

```
import java.io.*;
import java.lang.*;
import java.lang.annotation.*;
import java.lang.constant.*;
import java.lang.foreign.*;
import java.lang.invoke.*;
import java.lang.module.*;
import java.lang.ref.*;
import java.lang.reflect.*;
import java.lang.runtime.*;
import java.math.*;
import java.net.*;
import java.net.spi.*;
import java.nio.*;
import java.nio.channels.*;
import java.nio.channels.spi.*;
import java.nio.charset.*;
import java.nio.charset.spi.*;
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.nio.file.spi.*;
import java.security.*;
import java.security.cert.*;
import java.security.interfaces.*;
import java.security.spec.*;
import java.text.*;
import java.text.spi.*;
```

```
import java.time.*;
import java.time.chrono.*;
import java.time.format.*;
import java.time.temporal.*;
import java.time.zone.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;
import java.util.jar.*;
import java.util.random.*;
import java.util.regex.*;
import java.util.spi.*;
import java.util.stream.*;
import java.util.zip.*;
import javax.crypto.*;
import javax.crypto.interfaces.*;
import javax.crypto.spec.*;
import javax.net.*;
import javax.net.ssl.*;
import javax.security.auth.*;
import javax.security.auth.callback.*;
import javax.security.auth.login.*;
import javax.security.auth.spi.*;
import javax.security.auth.x500.*;
import javax.security.cert.*;
```

O

Module Import Declarations

```
import module java.base;
```

Imports ambiguos

```
import module java.desktop;
```

```
Element e = ...
```

```
// javax.swing.text.Element (interface)
```

```
// or
```

```
// javax.swing.text.html.parser.Element  
(class)
```

Imports ambiguos

```
import module java.desktop;  
import javax.swing.text.Element; //ambiguity resolved!
```

```
Element e = ...  
// javax.swing.text.Element (interface)
```

Compact Source Files and Instance Main Methods

JEP 512

Evolucionar el lenguaje Java para que los principiantes puedan escribir sus primeros programas sin tener que entender features diseñadas para programas grandes.

Sin usar un dialecto separado del lenguaje, los principiantes pueden escribir declaraciones simplificadas para programas de una sola clase y luego expandirlos sin fricción para usar features más avanzadas a medida que desarrollan sus habilidades.



Simple Source Files and Instance Main Methods

Mi primer programa

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Declaración de clase y modificador public

Parámetros para interactuar con el shell del sistema operativo

El modificador static es parte del modelo de clases y objetos

Importar clases utilitarias básicas

"Ignoren todo esto... lo entenderán después"

Permitir que los métodos main omitan public, static y String[] args

Mi primer programa

```
class HelloWorld {  
    void main() {  
        System.out.println("Hello, World!");  
    }  
}
```

Archivo fuente simple sin declaración de clase

Mi primer programa

```
void main() {  
    System.out.println("Hello, World!");  
}
```


Nueva clase: java.lang.IO

Mi primer programa

```
void main() {  
    IO.println("Hello, World!");  
}
```

Importar module.base automáticamente

Mi primer programa

```
import module.base
```

//Esta instrucción está implícita. No aparece realmente en el código.

```
void main() {  
    IO.println("Hello, World!");  
}
```

Importar automáticamente las clases e interfaces más comunes

Mi primer programa

```
import java.io.*;
import java.lang.*;
import java.lang.annotation.*;
import java.lang.constant.*;
import java.lang.foreign.*;
import java.lang.invoke.*;
import java.lang.module.*;
import java.lang.ref.*;
import java.lang.reflect.*;
import java.lang.runtime.*;
import java.math.*;
import java.net.*;
import java.net.spi.*;
import java.nio.*;
import java.nio.channels.*;
import java.nio.channels.spi.*;
import java.nio.charset.*;
import java.nio.charset.spi.*;
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.nio.file.spi.*;
import java.security.*;
import java.security.cert.*;
import java.security.interfaces.*;
import java.security.spec.*;
import java.text.*;
import java.text.spi.*;

import java.time.*;
import java.time.chrono.*;
import java.time.format.*;
import java.time.temporal.*;
import java.time.zone.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;
import java.util.jar.*;
import java.util.random.*;
import java.util.regex.*;
import java.util.spi.*;
import java.util.stream.*;
import java.util.zip.*;
import javax.crypto.*;
import javax.crypto.interfaces.*;
import javax.crypto.spec.*;
import javax.net.*;
import javax.net.ssl.*;
import javax.security.auth.*;
import javax.security.auth.callback.*;
import javax.security.auth.login.*;
import javax.security.auth.spi.*;
import javax.security.auth.x500.*;
import javax.security.cert.*;
```

//Estas instrucciones están implícitas.
// No aparecen realmente en el código.

```
void main() {
    IO.println("Hello, World!");
}
```

Flexible Constructor Bodies

JEP 513

En el cuerpo de un constructor, permitir que aparezcan instrucciones antes de una invocación explícita de constructor, es decir, `super(...)` o `this(...)`.

Estas instrucciones no pueden referenciar el objeto en construcción, pero sí pueden inicializar sus campos y realizar otros cálculos seguros

Flexible Constructor Bodies

Validar argumentos del constructor

```
public class PositiveBigInteger extends BigInteger {  
  
    public PositiveBigInteger(long value) {  
        super(value); // Trabajo potencialmente innecesario... pero debe ir primero  
        if (value <= 0)  
            throw new IllegalArgumentException("non-positive value");  
    }  
}
```

Flexible Constructor Bodies

Validar argumentos del constructor

```
public class PositiveBigInteger extends BigInteger {  
    public PositiveBigInteger(long value) {  
        if (value <= 0)  
            throw new IllegalArgumentException("non-positive value");  
        super(value);  
    }  
}
```

Flexible Constructor Bodies

Preparar argumentos del constructor de la superclase

```
public Sub(Certificate certificate)
    var publicKey = certificate.getPublicKey();
    if (publicKey == null) throw ...
    byte[] certBytes = switch (publicKey) {
        case RSAKey rsaKey -> ...
        case DSAPublicKey dsaKey -> ...
        default -> ...
    };
    super(certBytes);
}
```



¡No tiene que ser la primera línea!

2018 2019 2020 2021 2022 2023 2024 2025 2026 2027 2028 2029 2030 2031 2032 2033

Oracle JDK

2025-09-16



28

27

26

25 LTS

24

23

22

21 LTS

17 LTS

11 LTS

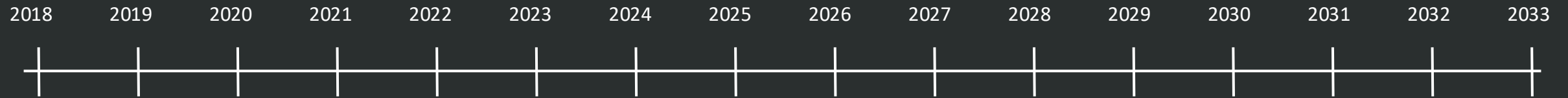
8



Free Use Java License



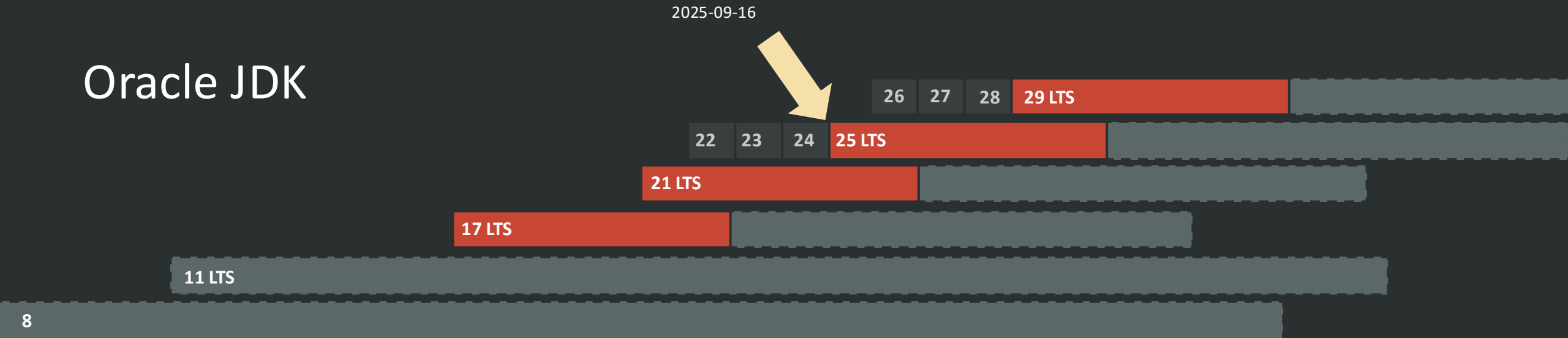
Gratis para desarrollo y uso personal; Suscripción u OCI para producción



2025-09-16



Oracle JDK



Free Use Java License



Gratis para desarrollo y uso personal; Suscripción u OCI para producción

Novedades en JDK 25

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]

Performance

Compact Object Headers [519]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Novedades en JDK 25 (para quienes vienen de JDK 21)

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Java más fácil de aprender

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]

Module Import Declarations [511]

Compact Source Files and Instance Main Methods [512]

Flexible Constructor Bodies [513]

Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]

ZGC: *[only]* Generational Mode [474,490]

Region Pinning for G1 [423]

Late Barrier Expansion for G1 [475]

Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]

Ahead-of-Time Method Profiling [515]

Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]

Permanently Disable the Security Manager [486]

Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]

Scoped Values [506]

Stable Values ^{Preview} [502]

Vector API 10th Incubator [508]

Foreign Function & Memory API [454]

Class File API [484]

Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]

PEM Encodings of Cryptographic Objects Preview [470]

Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]

Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]

JFR Cooperative Sampling [518]

JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]

Launch Multi-File Source-Code Programs [458]

Más fácil escribir aplicaciones

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]

Module Import Declarations [511]

Compact Source Files and Instance Main Methods [512]

Flexible Constructor Bodies [513]

Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]

ZGC: *[only]* Generational Mode [474,490]

Region Pinning for G1 [423]

Late Barrier Expansion for G1 [475]

Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]

Ahead-of-Time Method Profiling [515]

Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]

Permanently Disable the Security Manager [486]

Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]

Scoped Values [506]

Stable Values ^{Preview} [502]

Vector API 10th Incubator [508]

Foreign Function & Memory API [454]

Class File API [484]

Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]

PEM Encodings of Cryptographic Objects Preview [470]

Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]

Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]

JFR Cooperative Sampling [518]

JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]

Launch Multi-File Source-Code Programs [458]

Mejor performance en Garbage Collection

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Mejor performance en runtime (opt-in por ahora)

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Leyden: mejorar startup, tiempo hasta peak performance y footprint

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Mejoras en concurrencia

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Mejor profiling

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Eliminar cuellos de botella en el desarrollo de la plataforma

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Consentimiento informado para "no-guardrails"

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

Reforzar seguridad: algoritmos post-quantum

Lenguaje

Primitive Types in Patterns, instanceof, and switch 3rd Preview [507]
Module Import Declarations [511]
Compact Source Files and Instance Main Methods [512]
Flexible Constructor Bodies [513]
Unnamed Variables & Patterns [456]

Performance

Compact Object Headers [519]
ZGC: *[only]* Generational Mode [474,490]
Region Pinning for G1 [423]
Late Barrier Expansion for G1 [475]
Synchronize Virtual Threads without Pinning [491]

Leyden

Ahead-of-Time Command-Line Ergonomics [514]
Ahead-of-Time Method Profiling [515]
Ahead-of-Time Class Loading & Linking [483]

Futuras eliminaciones y restricciones

Prepare to Restrict the Use of JNI [472]
Permanently Disable the Security Manager [486]
Warn upon Use of Memory-Access Methods in `sun.misc.Unsafe` [498]

Librerías

Structured Concurrency ^{5th Preview} [505]
Scoped Values [506]
Stable Values ^{Preview} [502]
Vector API 10th Incubator [508]
Foreign Function & Memory API [454]
Class File API [484]
Stream Gatherers [485]

Librerías de Seguridad

Key Derivation Function API [510]
PEM Encodings of Cryptographic Objects Preview [470]
Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism [496]
Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm [497]

Monitoreo

JFR CPU-Time Profiling Experimental [509]
JFR Cooperative Sampling [518]
JFR Method Timing & Tracing [520]

Tools

Markdown Documentation Comments [467]
Launch Multi-File Source-Code Programs [458]

No relevante para Oracle JDK

Cambios en OpenJDK

Generational Shenandoah [521]

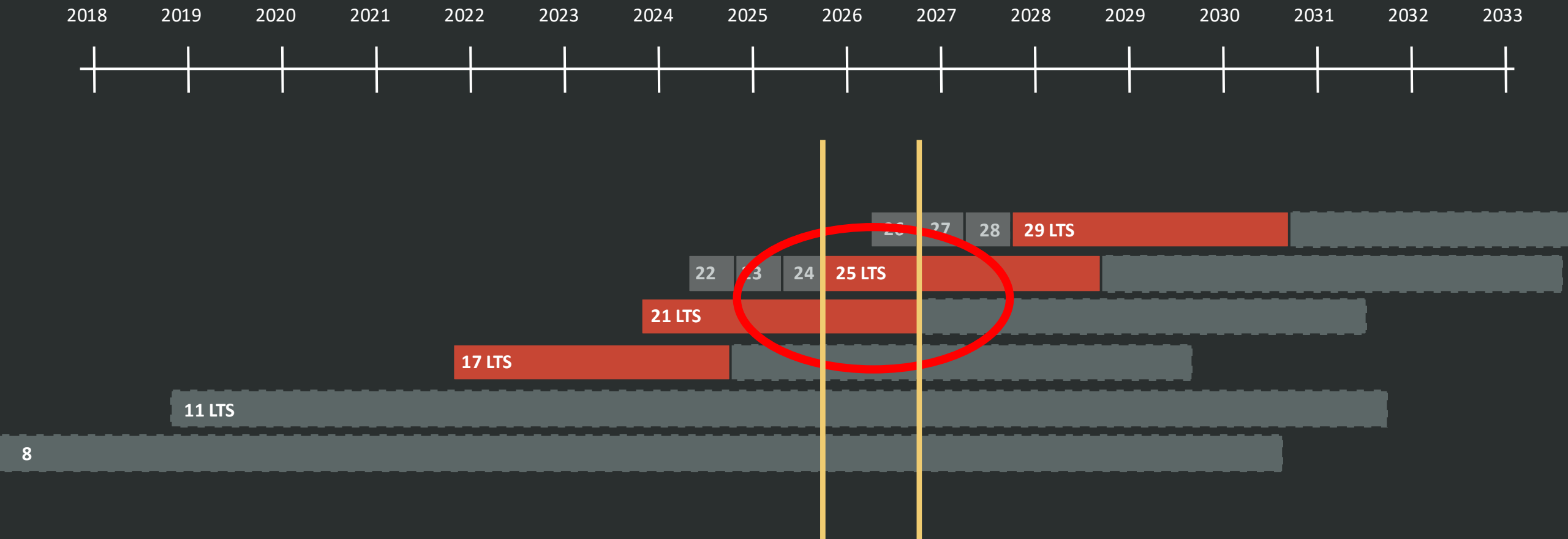
Remove the *[Linux]* 32-bit x86 Port [503]

Remove the Windows 32-bit x86 Port [479]

Linking Run-Time Images without JMODs [493]

La JEP faltante

String Templates^{2nd Preview} [459]



Cuenta regresiva de 1 año para el EoPL de JDK 21



Free Use Java License

Gratis para desarrollo y uso personal; Subscription u OCI para producción

Features principales (JEPs) en JDK 26

Lenguaje

Primitive Types in Patterns, instanceof, and switch ^{4th}
Preview [530]

Performance

G1 GC: Improve Throughput by Reducing
Synchronization [522]

Leyden

Ahead-of-Time Object Caching with Any GC [516]

Librerías

Prepare to Make final mean Final [500]

HTTP/3 for the HTTP Client API [517]

Lazy Constants ^{2nd preview} [526]

Structured Concurrency ^{6th Preview} [525]

Vector API ^{11th Incubator} [529]

Librerías de Seguridad

PEM Encodings of Cryptographic Objects ^{2nd Preview} [524]

Eliminaciones

Remove the Applet API [504]



Performance

Hacer más con menos

G1 GC: Improve Throughput by Reducing Synchronization

JEP 522

Aumentar el throughput de las aplicaciones al usar el garbage collector G1, reduciendo la sincronización requerida entre los threads de aplicación y los threads de GC.

G1 – La card table

G1 recupera memoria copiando objetos vivos del heap a nuevas regiones de memoria, lo que deja disponibles las regiones ocupadas previamente por esos objetos para asignar nuevos objetos. Las referencias a los objetos originales, almacenadas en campos de otros objetos en otras regiones, deben actualizarse para apuntar a sus nuevas copias. G1 rastrea las referencias entre regiones en una estructura llamada card table, que se actualiza cada vez que se almacena una referencia de objeto en un campo.

Algunas aplicaciones asignan objetos nuevos con mucha frecuencia. En esas aplicaciones, la card table puede crecer tanto que escanearla hace que G1 exceda su objetivo de pausa. Para evitarlo, G1 optimiza la card table en segundo plano mediante optimizer threads separados

Los optimizer threads deben sincronizarse con los application threads para evitar actualizaciones conflictivas de la card table. -> Complejo y, por lo tanto, lento.

Una segunda card table

JEP 522 introduce una segunda card table.

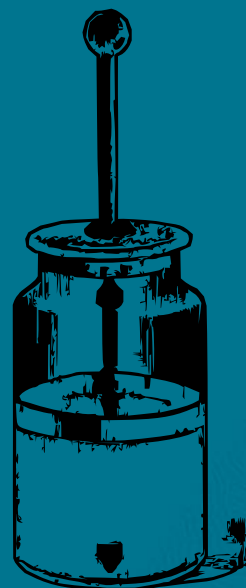
En este diseño, los application threads actualizan una card table sin sincronización, mientras los garbage collector threads procesan la otra. Periódicamente, se intercambian los roles de las dos tablas.

Este enfoque reduce el overhead de sincronización, simplifica las write barriers y mejora el throughput de la aplicación



De Project Leyden

Mejorar startup, tiempo hasta peak performance y footprint de programas Java



Ahead-of-Time Object Caching with Any GC

JEP 516

Mejorar el cache ahead-of-time, que permite a la HotSpot Java Virtual Machine mejorar el tiempo de startup y warmup, para que pueda usarse con cualquier garbage collector, incluido Z Garbage Collector (ZGC) de baja latencia. La implementación inicial de este cache AOT estaba estrechamente ligada a implementaciones específicas de GC, lo que la hacía incompatible con GCs como ZGC.

JEP 516 resuelve esta limitación desacoplando el cache AOT de los detalles específicos de GC. La mejora clave consiste en almacenar objetos Java cacheados en un formato neutral e independiente del GC. Este enfoque permite que la JVM cargue estos objetos secuencialmente en memoria, sin importar el GC en uso. Al eliminar la dependencia de formatos específicos de GC, el cache AOT se vuelve compatible con distintos GCs, incluido ZGC.



Librerías

Los trabajos más difíciles, comunes y tediosos

Prepare to make final mean Final

JEP 500

Emitir advertencias sobre usos de deep reflection para mutar campos final.

Estas advertencias preparan a los desarrolladores para una versión futura que garantice integrity by default al restringir la mutación de campos final, lo que hará que los programas Java sean más seguros y potencialmente más rápidos.

Los desarrolladores de aplicaciones pueden evitar tanto las advertencias actuales como las restricciones futuras habilitando selectivamente la capacidad de mutar campos final donde sea esencial.

Varias opciones para deshabilitar las advertencias (y eventualmente habilitar la mutación de campos final):

Opción de command line, variable de entorno, archivo de argumentos pasado al launcher, entradas de manifest en archivos JAR ejecutables, o programáticamente (por usuarios de una biblioteca).

HTTP/3 for the HTTP Client API

JEP 517

Actualizar la HTTP Client API para soportar el protocolo HTTP/3, de modo que bibliotecas y aplicaciones puedan interactuar con servidores HTTP/3 con cambios mínimos de código.

Objetivos

- Actualizar la HTTP Client API para enviar y recibir requests y responses HTTP/3.
- Requerir solo cambios menores en la HTTP Client API y en el código de aplicación.
- No cambiar la versión de protocolo predeterminada de HTTP/2 a HTTP/3; más bien, permitir que los desarrolladores opten por HTTP/3.

Lazy Constants (2nd Preview)

JEP 526

Introducir una API para lazy constants, que son objetos que contienen datos no modificables.

La JVM trata las lazy constants como constantes reales, habilitando las mismas optimizaciones de performance que se habilitan al declarar un campo final.

Sin embargo, en comparación con los campos final, las lazy constants ofrecen más flexibilidad sobre el momento de su inicialización.

Una lazy constant se inicializa en algún momento antes de recuperar su contenido por primera vez y, después de eso, es inmutable.

Preview en JDK 25 con el
nombre: Stable Values

Lazy Constants

Sin lazy constants debemos recurrir a campos final

```
class OrderController {  
  
    private final Logger logger = Logger.create(OrderController.class);  
  
    void submitOrder(User user, List<Product> products) {  
        logger.info("order started");  
        ...  
        logger.info("order submitted");  
    }  
  
}
```

\\Como logger es final, debe inicializarse eager
\\cuando se crea una instancia de OrderController.

Lazy Constants

Con lazy constants obtenemos lazy initialization manteniendo la inmutabilidad

```
class OrderController {  
  
    private final LazyConstant<Logger> logger = LazyConstant.of(  
        () -> Logger.create(OrderController.class)  
    )  
  
    void submitOrder(User user, List<Product> products) {  
        logger.get().info("order started");  
        ...  
        logger.get().info("order submitted");  
    }  
}
```

Structured Concurrency (6th Preview)

JEP 525

Structured concurrency trata grupos de tareas relacionadas que se ejecutan en distintos threads como unidades únicas de trabajo, simplificando el manejo de errores y la cancelación, mejorando la confiabilidad y aumentando la observabilidad.

El análogo concurrente de structured programming

- Cuando la ejecución se divide en varios flujos concurrentes, deben reunirse en el mismo bloque de código
- Idea simple, con implicaciones profundas
- Muchas de las cosas más difíciles se vuelven más fáciles (cancellation, shutdown)

Vector API (11th Incubator)

JEP 529

Expresar cálculos vectoriales que se compilen confiablemente en runtime a instrucciones vectoriales óptimas en CPUs soportadas, logrando así performance superior a cálculos escalares equivalentes.

Portable entre hardware que soporta distintos tamaños de vector.

En plataformas sin vectores, la degradación gradual produce código competitivo con loops desenrollados manualmente

Probablemente seguirá en preview hasta que
Project Panama entregue más mejoras



Librerías de Seguridad

Preparándonos para Post Quantum Cryptography

PEM Encodings of Cryptographic Objects (2nd Preview)

JEP 524

API para codificar objetos que representan cryptographic keys, certificates y certificate revocation lists en el formato de transporte ampliamente usado Privacy-Enhanced Mail (PEM), y para decodificar desde ese formato de vuelta a objetos. Es una preview API.

Una nueva interface y tres nuevas clases en el package java.security:

- DEREncodable interface
- clases PEMEncoder y PEMDecoder
- clase PEM, que implementa DEREncodable, para codificar y decodificar textos PEM

ejemplo de un objeto criptográfico codificado en PEM; en este caso, una public key de elliptic curve.

```
-----BEGIN PUBLIC KEY-----  
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEi/kRGOL7wCPTN4KJ2ppeSt5UYB6u  
cPjjjuKDtFTXbguOIFDdZ65O/8HTUqS/sVzRF+dg7H3/tkQ/36KdtuADbwQ==  
-----END PUBLIC KEY-----
```

2nd Preview en JDK 26



Eliminaciones

No se puede avanzar sin soltar el pasado

Remove the Applet API

JEP 504

Eliminar la Applet API, que fue deprecated for removal en JDK 17 (2021).

Es obsoleta porque ni las versiones recientes de JDK ni los navegadores actuales soportan applets.
Elimina:

- Todo el package `java.applet`
- Clases adicionales
 - `java.beans.AppletInitializer`
 - `javax.swing.JApplet`
- Elementos de API que referencian las clases e interfaces anteriores, incluidos métodos y campos en:
 - `java.beans.Beans`
 - `javax.swing.RepaintManager`